

T09 Security Testing

Quy trình kiểm thử an ninh cho EShop bằng SAST, DAST và AI triage có kiểm chứng.

Semgrep

OWASP ZAP

OpenAI / Gemini

DAST + SAST

Giảng viên hỗ trợ

● **Trần Duy Hoàng**
Giảng viên chính

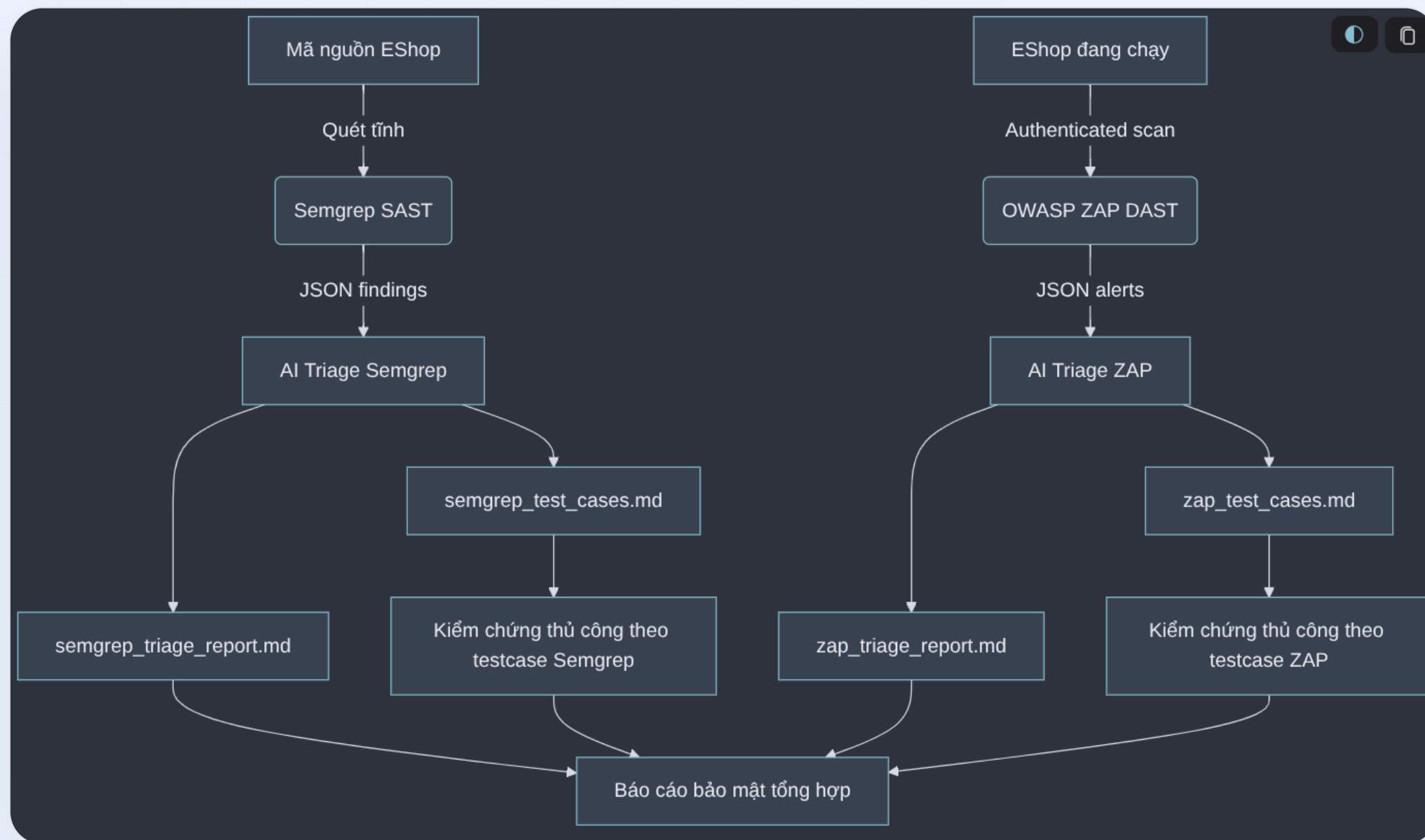
● **Hồ Tuấn Thanh**
Trợ giảng

Lê Trung Kiên
MSSV 23127075

Mai Thị Kim Duyên
MSSV 23127185

Lâm Hữu Khánh
MSSV 23127205

Lê Mai Hoài Bảo
MSSV 23127326



OWASP Top 10 2025 dùng làm khung chọn ruleset và scan policy.

A01

Broken Access Control

Kiểm soát quyền truy cập không chặt, user có thể truy cập dữ liệu hoặc chức năng ngoài quyền.

A02

Security Misconfiguration

Cấu hình server, framework, CORS, header hoặc môi trường triển khai không an toàn.

A03

Software Supply Chain Failures

Rủi ro từ thư viện, dependency, build pipeline, artifact hoặc thành phần bên thứ ba.

A04

Cryptographic Failures

Mã hóa, truyền dữ liệu, lưu secret hoặc bảo vệ dữ liệu nhạy cảm không đúng cách.

A05

Injection

Dữ liệu đầu vào bị đưa vào SQL, command, template hoặc interpreter mà không kiểm soát đúng.

A06

Insecure Design

Thiết kế nghiệp vụ hoặc kiến trúc thiếu cơ chế phòng vệ ngay từ đầu.

A07

Authentication Failures

Xác thực, token, session, reset password hoặc quản lý tài khoản có lỗi.

A08

Software or Data Integrity Failures

Không kiểm chứng tính toàn vẹn của dữ liệu, update, CI/CD hoặc code được nạp vào hệ thống.

A09

Logging and Alerting Failures

Thiếu log, monitoring hoặc cảnh báo khiến việc phát hiện sự cố bị chậm.

A10

Mishandling of Exceptional Conditions

Xử lý lỗi hoặc ngoại lệ không an toàn, làm lộ thông tin hoặc tạo trạng thái khó kiểm soát.

SAST đọc source trước khi app chạy.

Semgrep dùng rule pattern để bắt hardcoded secret, request HTTP không mã hóa, injection pattern và lỗi framework phổ biến.

01

Input

Mã nguồn và file cấu hình trong ./eshop-sut.

02

Ruleset

p/owasp-top-ten là flow chính theo đề T09.

03

Output

JSON finding thô để giữ lại làm evidence scan.

04

Triage

AI hỗ trợ đọc context, tester chốt kết luận.

Cài Semgrep CLI trước khi nói tới scan.

Phần triage dùng Python dependency riêng, còn việc quét source cần Semgrep CLI chạy được trong terminal.

Linux

```
python3 -m pip  
install semgrep
```

macOS

```
brew install  
semgrep
```

Windows

```
python -m pip  
install semgrep
```

Verify

Chạy `semgrep --version` trước khi scan.

```
# Tạo môi trường cho script triage  
python3 -m venv .venv  
source .venv/bin/activate  
python -m pip install -r src/semgrep/requirements.txt
```

```
# Cài và kiểm tra Semgrep CLI  
python3 -m pip install semgrep  
semgrep --version
```

Scan đúng source root, loại bỏ dependency và build output.

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -r src/semgrep/requirements.txt

# source mặc định
./eshop-sut
```

- **Chạy từ root repo**
Seminar-SoftwareTesting là working directory chuẩn.
- **Source khác vị trí**
Đặt SOURCE_ROOT trỏ tới source EShop thật.
- **Không quét nhiều**
Exclude node_modules, dist, build, .next.

Lệnh chính tạo evidence gốc theo OWASP Top 10.

```
semgrep scan --config "p/owasp-top-ten" \
  --exclude node_modules --exclude dist --exclude build --exclude .next \
  --json -o src/semgrep/output/semgrep_results.json ./eshop-sut
```

File

src/semgrep/output/semgrep_results.json

Đọc nhanh

File, line, rule ID, severity, CWE và OWASP mapping.

Nguyên tắc

Giữ JSON gốc để đối chiếu với triage report.

Scan mở rộng tăng coverage, không thay thế flow chính.

Sau ruleset bắt buộc, nhóm có thể quét thêm theo công nghệ của EShop để phân tích sâu hơn.

p/nodejs

Backend Node.js và Express-like patterns.

p/javascript

JS security rules phổ biến.

p/react

Frontend React và request/browser patterns.

Custom rules

src/semgrep/rules/eshop-security.yml

Output: semgrep_results_extended.json

Coverage analysis

AI triage cần cấu hình rõ, không phải magic report.

```
AI_PROVIDER=openai-compatible  
AI_MODEL=google/gemini-2.5-flash-lite  
AI_MAX_TOKENS=1800  
OPENROUTER_API_KEY=sk-or-v1-...  
OPENROUTER_BASE_URL=https://openrouter.ai/api/v1
```

Không commit key

src/semgrep/.env
chứa API key thật.

Khi hết credit

Giảm AI_MAX_TOKENS
hoặc chạy --offline.

Từ JSON finding thô sang report kiểm chứng được.

01

Input

semgrep_results.json

02

Context

Script lấy source context từ -
-source-root.

03

AI Triage

semgrep_ai_triage.py
phân tích từng finding.

04

Output

Report, test cases và folder
findings/.

```
python src/semgrep/semgrep_ai_triage.py \  
src/semgrep/output/semgrep_results.json \  
--source-root eshop-sut \  
--output-dir src/semgrep/output
```

True Positive / False Positive / Needs Human Review

AI hỗ trợ, tester quyết định

12

findings hiện có

Hardcoded JWT secret trong backend/server.js và nhiều HTTP request không mã hóa trong frontend-mobile/App.js.

Đừng đọc nhãn AI như kết luận cuối.

- **Code tự viết hay dependency?**
Loại bỏ build output và thư viện ngoài scope.
- **Dữ liệu có tới sink nguy hiểm?**
Finding chỉ có giá trị khi đường đi runtime hợp lý.
- **Secret hoặc URL có dùng thật?**
Phân biệt production, local lab, sample và test helper.

SEMGREP-001

Rule: javascript.jsonwebtoken.security.jwt-hardcode.hardcoded-jwt-secret

File: eshop-sut/backend/server.js:51

CWE-798 / OWASP A07 Authentication Failures

Hardcoded JWT secret là lỗi source-code có impact rõ.

Vì sao nguy hiểm?

Nếu secret bị lộ, attacker có thể tự ký JWT hợp lệ.

Ảnh hưởng demo

Endpoint cần auth có thể bị thử bằng forged token.

Evidence: backend/server.js

Authentication Failures

Finding chỉ có giá trị seminar khi có bước kiểm chứng.

```
GET http://localhost:3000/api/users/me
Authorization: Bearer <forged_token>
```

```
# PoC helper
node src/semgrep/exploit.js
```

TC-SEMGREP-001

Kiểm tra backend có chấp nhận JWT giả hay không.

Nếu 200 OK

Finding được xác nhận ở runtime.

Nếu 401/403

Ghi nhận không tái lập được hoặc đã bị chặn.

DAST kiểm thử ứng dụng đang chạy thật.

ZAP gửi request HTTP, crawl route, chạy passive scan và có thể active scan bằng payload kiểm thử.

Backend
localhost:3000

User FE
localhost:5173

Admin FE
localhost:5174

Cài ZAP gồm GUI, Python client và daemon cho CLI.

GUI dùng để quan sát request/response và cấu hình auth; CLI flow cần daemon tại localhost:8090.

Windows

```
winget install --  
id=ZAP.ZAP -e
```

macOS

```
brew install --  
cask zap
```

Linux

```
flatpak install  
flathub  
org.zaproxy.ZAP
```

Java

Cần Java 17+ nếu bản
cài không bundle Java.

Dependency cho script CLI

```
python3 -m venv .venv  
source .venv/bin/activate  
python -m pip install python-owasp-zap-v2.4
```

Chạy ZAP daemon bằng Docker

```
docker run --rm -d --name eshop-zap --network host \  
-v $(pwd):$(pwd) \  
ghcr.io/zaproxy/zaproxy:stable zap.sh -daemon \  
-port 8090 -host 0.0.0.0 -config api.disablekey=true
```


Setup ZAP là phần dễ làm sai nhất khi demo.

```
ZAP_TARGET=http://localhost:3000
ZAP_URL=http://localhost:8090
ZAP_AUTH_ROLE=none
ZAP_MAX_URLS=300
ZAP_REPORT_FORMAT=json
ZAP_REPORT_FILE=src/zap/output/backend_basic.json
```

```
# Credential test cho authenticated scan
ZAP_USER_EMAIL=test@eshop.com
ZAP_USER_PASSWORD=Test1234!
ZAP_ADMIN_EMAIL=admin@eshop.com
ZAP_ADMIN_PASSWORD=Admin123!
```

src/zap/.env

- **Daemon**
CLI cần ZAP daemon tại localhost:8090, có thể chạy Docker.
- **Scope**
Giới hạn target local 3000, 5173, 5174.
- **User account**
test@eshop.com / Test1234!
- **Admin account**
admin@eshop.com / Admin123!

Không commit credential thật

GUI dùng để nhìn request/response và cấu hình auth thủ công.

01

Public scan

Quick Start, Automated Scan, nhập target, Attack.

02

Proxy browser

Firefox qua localhost:8080 để bắt traffic thật.

03

Auth context

Flag POST /api/login làm JSON login request.

04

Forced user

Chạy Spider/AJAX Spider sau khi context đúng.

CLI public scan tạo JSON để pipeline xử lý tiếp.

```
python src/zap/scan_zap.py \  
  --target http://localhost:3000 \  
  --report-format json \  
  --output-file src/zap/output/backend_basic.json  
  
# Frontend SPA: thêm --ajax-spider
```

Backend public

backend_basic.json

Frontend public

Dùng AJAX Spider để khám phá route sau JS.

HTML khi đọc tay

JSON khi đưa vào AI triage.

Authenticated scan phải tách role và ngân sách URL.

```
python src/zap/scan_zap.py \  
  --target http://localhost:5173 \  
  --auth-role user \  
  --forced-user \  
  --ajax-spider \  
  --report-format json \  
  --output-file src/zap/output/frontend_user_basic.json
```

User

frontend_user_basic.json,
route user sau login.

Admin

frontend_admin_basic.json,
target 5174.

Backend API

Có thể scan 3000 với --
auth-role user.

401/403

Kiểm tra credential, login
endpoint và forced user.

Report chỉ có giá trị khi đúng target, đúng session và đúng scope.

Backend
src/zap/output/backend_basic.json

User FE
src/zap/output/frontend_user_basic.json

Admin FE
src/zap/output/frontend_admin_basic.json

KIỂM TRA		CÂU HỎI	LÝ DO
Target		Alert có thuộc EShop local không?	Tránh lẫn backend, frontend hoặc host ngoài scope.
Auth		Endpoint public hay cần session?	Impact khác nhau giữa guest, user và admin.
Evidence		Request/response đủ để kết luận chưa?	Local HTTP/dev server có thể tạo noise.

Gom nhiều JSON alert runtime thành Markdown report và testcase.

```
python src/zap/zap_ai_triage.py \  
  src/zap/output/backend_basic.json \  
  src/zap/output/frontend_user_basic.json \  
  src/zap/output/frontend_admin_basic.json \  
  --target-prefix http://localhost:3000 \  
  --target-prefix http://localhost:5173 \  
  --target-prefix http://localhost:5174
```

Report

zap_triage_report.md

Test cases

zap_test_cases.md

Per alert

alerts/*_prompt.md và
*_ai_output.md

CSP: Failure to Define Directive with No Fallback

Testcase trong report: TC-ZAP-130 đến TC-ZAP-134.

Evidence: Content-Security-Policy: default-src 'none'

CSP alert là ví dụ tốt cho evidence runtime.

Replay request

Gửi lại request ZAP ghi nhận và xem response header hiện tại.

Khuyến nghị

Bổ sung directive phù hợp: script-src, style-src, img-src, connect-src.

Tool và AI không thay thế trách nhiệm kỹ thuật của từng vai trò.

VAI TRÒ	OUTPUT	TRÁCH NHIỆM CHÍNH
Tester	Finding, testcase, evidence	Chạy tool, triage, tái hiện lỗi và phân loại trạng thái.
Developer	Patch, unit/integration test	Xác định root cause, sửa code/config, chống tái diễn.
SOC	Log, alert, incident note	Theo dõi production và xử lý dấu hiệu khai thác thật.

Các lỗi demo thường gặp đều liên quan tới môi trường, key hoặc scope.

Python package

Dùng `.venv`, tránh cài global vào system Python.

Sai source root

Kiểm tra `./eshop-sut` hoặc đặt `SOURCE_ROOT`.

OpenRouter 402

Hết credit, request quá tốn token, hoặc cần chạy `--offline`.

ZAP daemon

Kiểm tra Docker, image ZAP, hoặc daemon `localhost:8090`.

Auth 401/403

Kiểm tra credential, `/api/login`, `/api/users/me`.

Report lần target

Clear session/history hoặc tách output theo target.

“

Evidence đến từ tool, nhưng kết luận phải đến từ kiểm chứng.

SAST chỉ ra rủi ro trong source code. DAST xác nhận hành vi runtime. AI giúp tăng tốc đọc report, nhưng tester vẫn chịu trách nhiệm xác nhận bằng evidence.

Cảm ơn mọi người đã theo dõi

Nhóm 06 / KDBK

Q&A